

Atividade Prática

Projeto de Deep Learning para Demandas Reais

Detecção de Falhas em Impressão 3D

Efeito Espaguete

Sistema de visão computacional que classifica imagens capturadas durante a impressão 3D FDM, distinguindo impressões saudáveis de falhas do tipo Efeito Espaguete e recomendando a interrupção automática do processo.

Disciplina: Tópicos Especiais em Computação

Curso de Análise e Desenvolvimento de Sistemas (TADS)

Equipe: Amanda Iasmin · Izaque Nicolas · José Henrique · Matheus Ribeiro · Victor Sarrís · Sabrina Laís

Junho de 2026

Sumário

Resumo.....3

1. Definição do Problema.....4

2. Objetivo da Solução.....5

3. Base de Dados.....5

4. Modelagem em Deep Learning.....6

5. Avaliação dos Resultados.....8

6. Aplicabilidade Real.....10

7. Demonstração Funcional.....11

Resumo

Este projeto desenvolve uma solução de Deep Learning aplicada à visão computacional para um problema real de manufatura: a detecção automática de falhas em impressoras 3D de deposição de material fundido (FDM). O alvo é o chamado Efeito Espaguete, no qual o filamento passa a ser depositado no ar, sem aderir ao modelo, inutilizando a peça e desperdiçando horas de impressão e material. A solução classifica imagens capturadas durante a impressão em duas classes, impressão saudável (success) e falha (failure), e, ao detectar falha, recomenda a interrupção imediata do processo.

Como evidência concreta, treinou-se um classificador binário de imagens por Transfer Learning sobre a arquitetura EfficientNetB3, em duas fases (extração de características e fine-tuning profundo). No conjunto de teste, o modelo alcançou acurácia de 85,7%, F1-score de 82,8% e ROC-AUC de 0,967, indicando excelente capacidade de separação entre as classes. A solução é entregue como uma aplicação web funcional de ponta a ponta: um backend Django que expõe o modelo por uma API de inferência e um frontend React que permite enviar uma foto e receber o diagnóstico com o respectivo grau de confiança.

Nota sobre esta documentação. Este é o relatório técnico consolidado do projeto. As Seções 1 a 7 reúnem, com figuras e tabelas, todo o conteúdo necessário à avaliação: definição do problema, objetivo, base de dados, modelagem, resultados, aplicabilidade e demonstração funcional.

1. Definição do Problema

1.1 Contexto do problema

A impressão 3D por deposição de material fundido (FDM) é amplamente utilizada em prototipagem, manufatura distribuída e produção acadêmica. O processo é, porém, suscetível a falhas silenciosas, em especial o Efeito Espaguete, em que o filamento é extrudado no ar sem aderir ao modelo, resultando em uma malha caótica de plástico que inutiliza completamente a peça. Como uma impressão pode durar de várias horas a dias, uma falha que ocorre no início e passa despercebida desperdiça todo o tempo e o material restantes.

1.2 Impacto do problema

Falhas desse tipo geram desperdício de filamento (que pode custar dezenas de reais por carretel), perda de tempo em impressões longas que continuam mesmo já comprometidas, risco de dano ao equipamento caso o filamento se acumule e obstrua componentes mecânicos e, em contextos de produção em série, impacto financeiro ampliado. O volume de impressões executadas sem supervisão presencial torna a triagem manual inviável, sobretudo quando várias máquinas operam simultaneamente.

1.3 Quem é afetado

São afetados makers, engenheiros, laboratórios de pesquisa, empresas de prototipagem e qualquer usuário de impressoras FDM de mesa aberta, especialmente aqueles que executam impressões longas sem acompanhamento contínuo, situação em que uma falha não detectada se traduz diretamente em prejuízo.

1.4 Por que o problema é relevante

A detecção automática de falhas elimina a necessidade de monitoramento humano constante, permite a interrupção imediata da impressão e reduz de forma significativa as perdas de material e de tempo. Em ambientes com várias impressoras, o ganho de escala é ainda maior, pois um único sistema de visão pode supervisionar todas as máquinas a custo marginal baixo.

1.5 Limitações da solução proposta

- O modelo atua sobre imagens estáticas, sem análise temporal de quadros consecutivos.
- A câmera precisa de posicionamento e iluminação adequados para captura confiável.
- O dataset é relativamente pequeno (327 imagens), o que pode limitar a generalização.
- A solução detecta apenas o Efeito Espaguete, e não outros tipos de falha (deslocamento de camada, sub-extrusão, etc.).
- O sistema é uma ferramenta de apoio: pressupõe a possibilidade de um humano no circuito para a decisão final de abortar.

1.6 Justificativa do uso de Deep Learning

A detecção visual do Efeito Espaguete é uma tarefa em que abordagens baseadas em regras explícitas tendem a falhar: a falha não tem um formato único nem previsível, manifestando-se como um emaranhado irregular cuja aparência varia com a peça, a câmera e a iluminação. Redes Neurais Convolucionais aprendem diretamente dos pixels os padrões discriminativos, sem necessidade de

regras manuais. Combinadas com Transfer Learning, têm desempenho consolidado em classificação de imagens mesmo com conjuntos de dados pequenos, o que as torna a escolha adequada para este problema.

2. Objetivo da Solução

2.1 O que o sistema pretende fazer

Desenvolver, documentar e demonstrar uma solução de Deep Learning capaz de classificar automaticamente imagens capturadas durante a impressão 3D em duas categorias mutuamente exclusivas, impressão saudável (success) e Efeito Espaguete (failure), e de, em caso de falha, sinalizar um alerta recomendando a interrupção imediata da impressão pelo G-code M112 (parada de emergência).

2.2 Qual resultado espera produzir

Um classificador binário de imagens robusto, embarcado em uma aplicação web funcional, que recebe uma foto da mesa de impressão e devolve o diagnóstico acompanhado do grau de confiança do modelo. O resultado deve ser confiável o suficiente para apoiar a decisão de abortar uma impressão sem supervisão humana presencial.

2.3 Qual parte do problema será solucionada

O projeto resolve a etapa de classificação automática (a triagem visual da impressão), que é o núcleo da demanda. A captura periódica das imagens e a execução efetiva do comando de parada na impressora são tratadas como integrações externas: a solução fornece o diagnóstico e a recomendação, deixando a atuação física a cargo do operador ou de uma camada de automação.

2.4 Qual ganho esperado a solução oferece

Ganho econômico (redução de desperdício de filamento e de tempo de máquina), ganho de autonomia (monitoramento de impressões longas sem presença humana) e ganho ambiental (menos descarte de plástico por peças com falha). Em escala, o mesmo sistema pode supervisionar múltiplas impressoras, ampliando o retorno sobre um investimento mínimo em hardware de câmera.

3. Base de Dados

Origem, tipo e quantidade

Utilizou-se o conjunto público 3D Printing - SUCCESS - FAILURE DATASET (finetuned), disponível no Kaggle e derivado do projeto open-source Obico / The Spaghetti Detective, que coleta imagens reais de impressoras FDM de mesa aberta ao redor do mundo. O tipo de dado é imagem fotográfica da mesa de impressão. O total é de 327 imagens, rotuladas em duas classes (success e failure), com diversidade de modelos de impressora, ângulos de câmera e condições de iluminação.

Estratégia de coleta

A base é pública e já organizada em pastas por classe e por split, o que dispensa coleta primária ou anotação manual. As imagens foram obtidas diretamente do repositório do dataset e carregadas a partir da estrutura de diretórios (data/train, data/valid, data/test), garantindo reprodutibilidade.

Estratégia de tratamento e pré-processamento

O pipeline de tratamento padroniza todas as imagens para 300×300 pixels, tamanho usado pelo modelo em produção, por meio de `image_dataset_from_directory`, com rótulo binário (`label_mode='binary'`). A normalização dos pixels é realizada internamente pela própria EfficientNetB3, consistente com os pesos pré-treinados na ImageNet. O carregamento é otimizado com `prefetch` e `tf.data.AUTOTUNE`. Durante o treino, aplica-se Data Augmentation (inversão horizontal e vertical, rotação de até 20% e zoom de até 20%) para ampliar artificialmente a variabilidade e reduzir overfitting.

Balanceamento

O dataset é balanceado nos conjuntos de treino e validação (114 e 32 amostras por classe, respectivamente), o que dispensa técnicas de reamostragem ou pesos de classe. No conjunto de teste há leve desbalanceamento (20 falhas contra 15 sucessos), considerado na leitura das métricas.

Distribuição e separação treino / validação / teste

Os dados já vêm divididos em três conjuntos disjuntos, conforme a tabela a seguir:

Conjunto	failure	success	Total
Treino (train/)	114	114	228
Validação (valid/)	32	32	64
Teste (test/)	20	15	35
Total	166	161	327

Limitações da base

A base é relativamente pequena (327 imagens), o que pode limitar a generalização para impressoras e condições muito distintas das observadas no treino. As imagens foram capturadas em condições variadas de iluminação e ângulo, há possível viés para determinados modelos e marcas, e o conjunto cobre apenas o Efeito Espaguete, sem representar outros tipos de falha.

4. Modelagem em Deep Learning

4.1 Estratégia geral

A modelagem adota Transfer Learning sobre a EfficientNetB3 pré-treinada na ImageNet, em duas fases de ambição crescente: primeiro treina-se apenas a cabeça de classificação com a base congelada (extração de características); depois descongelam-se as últimas camadas da base para um fine-tuning profundo com taxa de aprendizado reduzida. Essa progressão adapta rapidamente o classificador ao novo domínio sem destruir o conhecimento visual já aprendido pela rede.

4.2 Tipo de rede e justificativa

A arquitetura escolhida é uma Rede Neural Convolutacional (CNN), na variante EfficientNetB3. A justificativa é dupla: CNNs são o padrão consolidado para classificação de imagens, e a família EfficientNet oferece excelente equilíbrio entre acurácia e custo computacional, viabilizando inferência em ambientes com recursos limitados. Sobre a base convolutacional (com `include_top=False`) foi

acoplada uma cabeça customizada, composta por GlobalAveragePooling2D, seguida de Dropout de 30% e de uma camada Dense com ativação sigmoide, adequada à classificação binária.

4.3 Função de ativação

Internamente, a EfficientNetB3 utiliza a ativação Swish em suas camadas convolucionais. Na camada de saída aplica-se a função sigmoide, que comprime a saída no intervalo de 0 a 1 e a interpreta como a probabilidade da classe success; o limiar de decisão é 0,5.

4.4 Função de perda

Adotou-se a entropia cruzada binária (Binary Crossentropy), apropriada para problemas de classificação com duas classes e saída sigmoide. Como o conjunto de treino é balanceado, não foi necessário ponderar as classes na função de perda.

4.5 Otimizador

Utilizou-se o otimizador Adam. Na Fase 1, com taxa de aprendizado de 0,001, para treinar rapidamente a cabeça de classificação; na Fase 2, com taxa reduzida a 1e-5, para ajustar finamente os filtros descongelados da base sem provocar esquecimento catastrófico.

4.6 Métricas de avaliação

As métricas adotadas são acurácia, precisão, revocação (recall), F1-score e ROC-AUC. Em um cenário de detecção de falhas, a revocação da classe failure é particularmente importante, pois um falso negativo (falha não detectada) é o erro mais custoso. O ROC-AUC resume a capacidade de separação entre as classes independentemente do limiar.

4.7 Quantidade de épocas e batch size

A Fase 1 foi treinada por 10 épocas e a Fase 2 por até 15 épocas, com Early Stopping interrompendo o treino quando a perda de validação deixava de melhorar. O batch size foi de 32 imagens, e o tamanho de entrada, 300×300 pixels.

4.8 Estratégias contra overfitting

Foram combinadas quatro estratégias: Data Augmentation (flip, rotação e zoom) integrado ao pipeline de treino; Dropout de 30% na camada densa; Early Stopping com restauração dos melhores pesos (restore_best_weights); e congelamento progressivo das camadas, com a base totalmente congelada na Fase 1 e descongelamento apenas parcial (últimas 30 camadas) na Fase 2.

4.9 Como o modelo é treinado

O treino é executado no notebook Deteccao_Falhas_Impressao3D.ipynb. Na Fase 1, com a base congelada, compila-se o modelo com Adam (lr=0,001) e treina-se a cabeça por 10 épocas. Na Fase 2, descongelam-se as últimas 30 camadas, recompila-se com Adam (lr=1e-5) e treina-se com os callbacks EarlyStopping(patience=4, monitor='val_loss') e ModelCheckpoint(monitor='val_accuracy', save_best_only=True). O melhor modelo é salvo em melhor_modelo_efficientnet_pc.keras, artefato carregado pelo backend em produção.

5. Avaliação dos Resultados

Esta seção apresenta os resultados quantitativos no conjunto de teste (35 amostras nunca vistas durante o treino) e a interpretação crítica do desempenho do modelo.

5.1 Resultado quantitativo

A tabela abaixo resume as métricas obtidas no conjunto de teste:

Métrica	Valor	Interpretação
Accuracy	85,71%	Proporção total de acertos
Precision	85,71%	Confiabilidade das previsões positivas
Recall	80,00%	Capacidade de encontrar os casos relevantes
F1-Score	82,76%	Equilíbrio entre precisão e revocação
ROC-AUC	0,967	Capacidade de separação entre as classes

5.2 Matriz de confusão

O modelo acertou 18 de 20 casos de falha (90% de detecção) e 12 de 15 casos de impressão saudável. Os 2 falsos negativos (falhas não detectadas) são o cenário mais crítico, pois permitiriam que a impressão continuasse defeituosa; os 3 falsos positivos (impressões saudáveis classificadas como falha) geram interrupções desnecessárias, porém de menor custo material.

	Predito: Falha	Predito: Sucesso
Real: Falha	18 (acerto)	2 (erro)
Real: Sucesso	3 (erro)	12 (acerto)

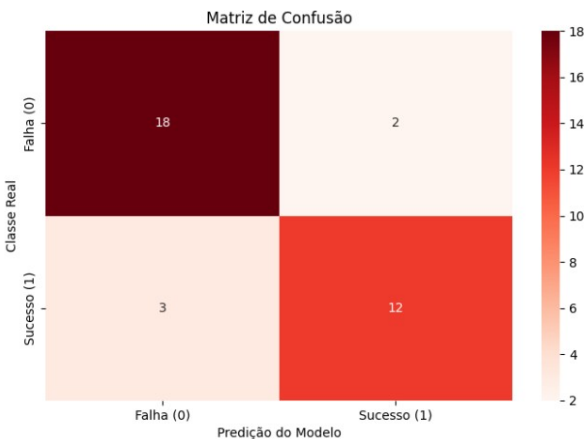


Figura 1: Matriz de confusão no conjunto de teste.

5.3 Curva ROC e curvas de aprendizado

A acurácia de validação atingiu cerca de 98% já nas primeiras épocas da Fase 1, mantendo-se estável durante o fine-tuning, enquanto a perda de validação convergiu para valores próximos a 0,15. O pico de perda de treino no início da Fase 2 é esperado, decorrente do descongelamento das camadas e da

redução da taxa de aprendizado, normalizando rapidamente em seguida. A curva ROC, com AUC de 0,967, confirma a forte separabilidade entre as classes.

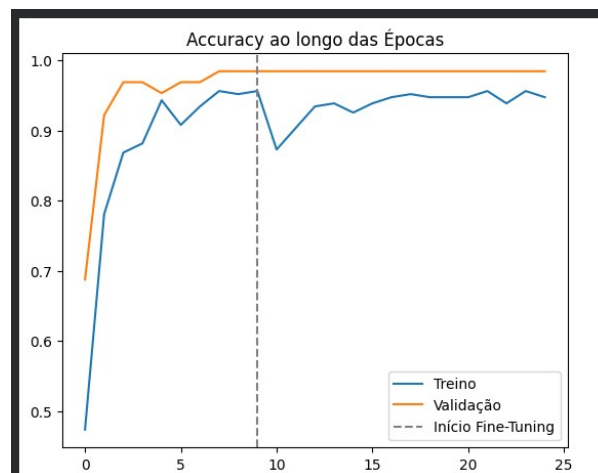


Figura 2: Acurácia de treino e validação ao longo das épocas.

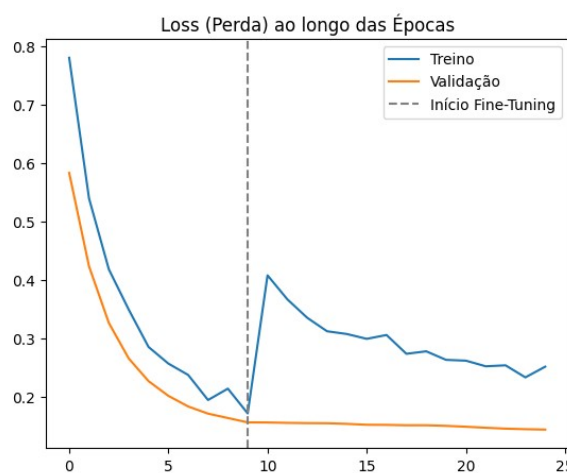


Figura 3: Perda (loss) de treino e validação ao longo das épocas.

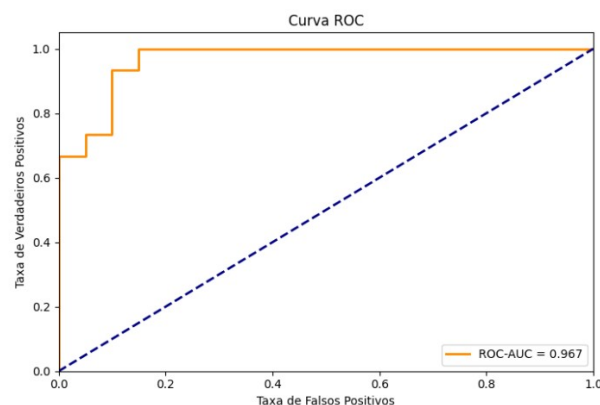


Figura 4: Curva ROC do modelo (AUC = 0,967).

5.4 Interpretação crítica

O ROC-AUC de 0,967 indica excelente capacidade de separação entre as classes. A acurácia de validação (cerca de 98%) é superior à de teste (85,71%), o que é esperado dado o tamanho reduzido do conjunto de teste (35 amostras), em que cada erro tem peso proporcionalmente maior. Para uso real, o ROC-AUC elevado é o indicador mais confiável, pois mede a separabilidade independentemente

do limiar de decisão. O ponto de atenção operacional são os 2 falsos negativos: como o custo de uma falha não detectada é alto, um ajuste de limiar para favorecer a revocação da classe failure pode ser desejável conforme o cenário de uso.

5.5 Limitações do modelo

- O dataset pequeno pode limitar a generalização para impressoras e condições muito diferentes das do treino.
- O modelo não detecta outros tipos de falha além do Efeito Espaguete.
- O desempenho pode cair sob iluminação muito distinta da observada nas imagens de treino.
- A avaliação se apoia em apenas 35 amostras de teste, o que aumenta a variância das métricas pontuais.

5.6 Possíveis melhorias futuras

- Ampliar o dataset com maior diversidade de impressoras, filamentos e condições de iluminação.
- Implementar análise temporal (sequência de quadros) para detecção mais robusta e menos sujeita a falsos positivos pontuais.
- Suportar múltiplas classes de falha (deslocamento de camada, sub-extrusão, entre outras).
- Converter o modelo para TensorFlow Lite, viabilizando inferência embarcada (Raspberry Pi, ESP32-CAM).
- Integrar diretamente com firmwares de impressora (Klipper, Marlin) para interrupção automática.

6. Aplicabilidade Real

6.1 Como a solução poderia ser usada no mundo real

O sistema pode ser integrado a qualquer impressora 3D FDM por meio de uma câmera de baixo custo conectada a um servidor local. A cada intervalo de tempo, uma foto da mesa é capturada, enviada ao backend e classificada pelo modelo. Detectada uma falha, a interface exibe o diagnóstico e a recomendação de executar o G-code M112. Em uma evolução natural, esse comando poderia ser disparado automaticamente, fechando o ciclo de monitoramento autônomo.

6.2 Limitações operacionais

O desempenho depende de uma câmera bem posicionada e de iluminação estável; mudanças bruscas de cenário podem exigir recalibração ou retreino. Como o modelo analisa quadros isolados, ruídos momentâneos podem gerar falsos positivos, mitigáveis por confirmação em múltiplos quadros. A solução cobre apenas o Efeito Espaguete e requer o backend em execução para funcionar.

6.3 Custos computacionais

O treino por Transfer Learning é viável em uma GPU de consumo, em ordem de minutos, dado o tamanho reduzido da base. A inferência é leve: a EfficientNetB3 classifica uma imagem em fração de segundo mesmo em hardware modesto, o que torna a solução compatível com servidores locais baratos e, futuramente, com dispositivos embarcados.

6.4 Possíveis impactos sociais, técnicos e econômicos

No plano econômico, a redução de desperdício de filamento e de tempo de máquina é direta. No técnico, a arquitetura desacoplada permite integrar o sistema a ecossistemas de automação existentes. No social e ambiental, democratiza-se o uso autônomo e confiável da impressão 3D e reduz-se o descarte de plástico decorrente de peças falhas.

6.5 Viabilidade prática da implementação

Alta. A base de dados é aberta, o modelo pré-treinado é gratuito, o pipeline é reproduzível e o custo de hardware é acessível. A entrega já inclui uma aplicação web funcional, o que demonstra a viabilidade da solução fim a fim e encurta o caminho para uma eventual adoção real.

7. Demonstração Funcional

O projeto entrega uma aplicação web completa, dividida entre um backend de inferência e um frontend de interação, além do notebook de treino que reproduz o modelo.

7.1 Interface web interativa

O frontend, construído em React 19 com Vite, apresenta o Monitor de Anomalias FDM. O usuário envia uma foto da mesa de impressão por clique ou arrastar-e-soltar (formatos JPG, PNG ou WEBP), visualiza uma pré-visualização e aciona a análise. O resultado é exibido com indicação visual (verde para impressão estável, vermelho para anomalia), o percentual de confiança do modelo e uma barra de progresso proporcional; em caso de falha, destaca-se a recomendação de abortar a impressão pelo G-code M112. A interface também trata erros, informando explicitamente quando o backend não está em execução.

7.2 Backend, API e modelo

O backend Django carrega o modelo treinado uma única vez, na inicialização, e expõe o endpoint POST /predict, que recebe a imagem em multipart/form-data (campo file). A view abre a imagem com Pillow, converte para RGB, redimensiona para 300×300, gera o tensor com NumPy, executa model.predict(), aplica o limiar de 0,5 e devolve um JSON com o status, a confiança e o indicador booleano is_sucesso. O CORS está habilitado para permitir o consumo direto pelo frontend em desenvolvimento.

Exemplo de resposta da API:

```
{ "status": "ALERTA: Falha Detectada (Efeito Espaguete)", "confianca": 94.37, "is_sucesso": false }
```

7.3 Como executar

O fluxo completo está descrito no README do repositório. Em resumo:

1. Obter o dataset no Kaggle e extraí-lo em data/train, data/valid e data/test.
2. Treinar o modelo executando o notebook Deteccao_Falhas_Impressao3D.ipynb a partir de backend/ (gera o arquivo .keras).
3. Iniciar o backend: python manage.py runserver (sobe em http://localhost:8000 e carrega o modelo).
4. Iniciar o frontend: npm install e npm run dev (acessível em http://localhost:5173).

5. Enviar uma foto pela interface e conferir o diagnóstico retornado pela API.